

Università degli studi di Urbino Carlo Bo

Corso di laurea in Informatica Scienza e Tecnologia

Progetto d'esame di Programmazione Logica e Funzionale

Anno accademico 2025/2026

Andrea Pedini, N. 322918

Matteo Fraternali, N. 316637

1 Specifica del Problema

Scrivere un programma Haskell e un programma Prolog che leggano da file e stampino a schermo i seguenti dati di input:

- Un insieme di numeri interi
- Un insieme di coppie di numeri interi appartenenti all'insieme precedente

Successivamente il programma dovrà:

- Costruire un grafo orientato i cui vertici sono costituiti dalla lista di interi letta da file e gli archi sono rappresentati dalle coppie di interi letti da file.
- Costruire un nuovo grafo orientato in cui i vertici sono le componenti fortemente connesse del grafo iniziale rappresentate da un singolo vertice scelto arbitrariamente tra i vertici di una componente fortemente connessa e gli archi sono i collegamenti tra le componenti fortemente connesse distinte.
- Acquisire un numero intero tra i vertici del grafo a rappresentare il vertice di partenza e stampare a schermo il numero di componenti fortemente connesse diverse dalla componente contenente il vertice di partenza che hanno grado entrante uguale a 0.

2 Analisi del problema

2.1 Dati di ingresso del problema

I dati di ingresso sono i seguenti:

- Un insieme di numeri interi
- Un insieme di coppie di numeri interi appartenenti all'insieme precedente
- Un numero intero

2.2 Dati di uscita del problema

I dati di uscita del problema sono costituiti dal grafo e le sue componenti fortemente connesse e dal numero di componenti fortemente connesse diverse dalla componente contenente il vertice di partenza che hanno grado entrante uguale a 0.

2.3 Relazioni intercorrenti tra i dati del problema

Le relazioni intercorrenti tra i dati di ingresso e i dati di uscita del problema riguardano la struttura del grafo e la sua trasformazione attraverso il concetto di componenti fortemente connesse (SCC - Strongly Connected Components).

Una componente fortemente connessa C è un insieme di vertici tale per cui per ogni v_1, v_2 appartenenti a C esiste un percorso da v_1, v_2 e uno da v_2, v_1 .

Il calcolo delle componenti fortemente connesse viene effettuato tramite l'algoritmo di Kosaraju. L'algoritmo si articola in due fasi principali basate su visite in profondità. Nella prima fase si esegue una visita in profondità sul grafo originale calcolando l'ordine di completamento dei vertici. In altre parole si compone una lista come risultato parziale, aggiungendovi i vertici uno alla volta solo quando sono stati visitati tutti i loro vertici adiacenti. Successivamente si costruisce il grafo trasposto, ottenuto invertendo la direzione di tutti gli archi. Nella seconda fase si esegue una nuova visita in profondità sul grafo trasposto, visitando i vertici estratti uno alla volta dalla lista di completamento composta nella prima fase. Ogni nuova visita individua esattamente una componente fortemente connessa. L'algoritmo ha complessità lineare $O(V + E)$, dove V è il numero dei vertici e E è il numero degli archi.

Una volta determinate le componenti fortemente connesse, oltre ad aver fornito come dati di uscita il grafo originale e le componenti calcolate, il dato di uscita principale sarà dato semplicemente dal numero delle SCC con grado entrante 0, escludendo la SCC del vertice di partenza.

Adottiamo questo metodo perchè siamo interessati a ottenere il numero minimo di archi da aggiungere per rendere tutto il grafo raggiungibile dal vertice scelto, mentre se ci limitassimo ad aggiungere tanti archi quanti sono i vertici non raggiungibili non otterremmo il numero minimo.

3 Progettazione dell'Algoritmo

3.1 Scelte di Progetto

Le scelte di progetto che sono state effettuate riguardano:

- la rappresentazione dei dati del grafo come semplici liste di interi per i vertici e liste di coppie di interi per gli archi.
- la definizione di visita in profondità generica che tramite un parametro formale consente di essere riutilizzata per effettuare le due visite in profondità previste dall'algoritmo di Kosaraju, che producono come risultato liste in cui i vertici vengono aggiunti al momento della visita o del completamento.

3.2 Passi dell'Algoritmo

- Acquisizione del grafo
 - Leggere da un file una lista di vertici e una lista di archi, costruendo un grafo diretto
- Visualizzazione iniziale
 - Stampa a schermo dei vertici e degli archi del grafo così come sono stati letti
- Calcolo delle componenti fortemente connesse (SCC)
 - Esaminare il grafo per identificare insiemi di vertici tali che ciascun vertice del gruppo sia raggiungibile da ogni altro vertice dello stesso gruppo
 - Generare una lista di questi insiemi (componenti)
 - Visualizzare le componenti trovate
- Costruzione del grafo compresso
 - Considerare ogni componente come un singolo vertice
 - Creare archi tra componenti se esiste almeno un arco nel grafo originale che collega vertici appartenenti a componenti diverse
 - Mostrare il grafo risultante, dove ogni vertice rappresenta una componente
- Calcolo del grado entrante delle componenti
 - Per ogni componente, contare quanti archi entrano in essa da altre componenti
 - Visualizzare questi valori
- Scelta del vertice di partenza
 - Richiedere all'utente di indicare un vertice specifico validando che esso sia presente nel grafo iniziale
 - Identificare la componente che contiene questo vertice
- Conteggio delle componenti indipendenti
 - Contare quante componenti nel grafo compresso non hanno archi in ingresso escludendo la componente contenente il vertice di partenza
 - Visualizzare il numero risultante.

4 Implementazione dell'Algoritmo

4.1 Implementazione in Haskell

```
-- #####
-- #          Corso di Programmazione Logica e Funzionale          #
-- #          Progetto per la sessione invernale A.A 2025/2026    #
-- #                      di Andrea Pedini                        #
-- #                      Matricola: 322918                        #
-- #                      e Matteo Fraternali                     #
-- #                      Matricola: 316637                        #
-- #####

{-
  Scrivere un programma Haskell e un programma Prolog che leggano
  da file e stampino a schermo i seguenti dati di input:
  * Un insieme di numeri interi
  * Una insieme di coppie di numeri interi
  Successivamente il programma dovrà:
  * Costruire un grafo orientato i cui vertici sono costituiti
    dalla lista di interi letta da file e gli archi
    sono rappresentati dalle coppie di interi letti da file.
  * Costruire un nuovo grafo orientato in cui i vertici sono le componenti
    fortemente connesse del grafo iniziale rappresentate da un singolo vertice
    scelto arbitrariamente tra i vertici di una componente fortemente connessa e
    gli archi sono i collegamenti tra le componenti fortemente connesse distinte.
  * Acquisire un numero intero tra i vertici del grafo a rappresentare il vertice di
    partenza e stampare a schermo il numero di componenti fortemente
    connesse diverse dalla componente contenente il vertice di partenza che
    hanno grado entrante uguale a 0.
-}

module Main where

-- Import della funzione nub per eliminare duplicati da una lista
import Data.List ( nub )
import System.IO
import GHC.IO.Encoding (setLocaleEncoding, utf8)

-----
-- FUNZIONE PRINCIPALE
-----

{-
  Azione principale del programma.

  1) Legge un grafo orientato dal file "input.txt".
  2) Verifica che il formato e il contenuto del grafo siano validi.
  3) Calcola le Componenti Fortemente Connesse (SCC) con Kosaraju.
  4) Costruisce il grafo compresso (ogni SCC diventa un nodo).
```

- 5) Richiede all'utente un vertice valido di partenza.
- 6) Conta quante SCC hanno grado entrante zero, escludendo la SCC contenente il vertice scelto.

Se il file contiene errori il programma termina in modo controllato.

```
-}
main :: IO ()
main = do
    risultato <- leggiGrafoDaFile "input.txt"

    case risultato of
        Nothing -> do
            putStrLn ("\nIl programma è stato terminato " ++
                "a causa di errori nel file di input.\n")

        Just (vertici, archi) -> do

            putStrLn "\n=====
            putStrLn "          GRAFO LETTO DA FILE          "
            putStrLn "-----"
            putStrLn $ "Vertici: " ++ show vertici
            putStrLn $ "Archi:   " ++ show archi

            let componenti = kosaraju vertici archi

            putStrLn "\n=====
            putStrLn "          COMPONENTI FORTEMENTE CONNESSE          "
            putStrLn "-----"
            mapM_
                (\(i, c) -> putStrLn $ "SCC " ++ show i ++ ": " ++ show c)
                (zip [0 ..] componenti)

            let grafoCompresso = comprimiGrafo componenti archi

            putStrLn "\n=====
            putStrLn "          GRAFO COMPRESSO          "
            putStrLn "-----"
            mapM_
                (\(i, j) -> putStrLn $ "SCC_" ++ show i ++ " -> SCC_" ++ show j)
                grafoCompresso

            verticePartenza <- acquisisciVertice vertici

            let numeroZero =
                contaSCCConGradoZero verticePartenza componenti grafoCompresso

            putStrLn "\n=====
            putStrLn $
                "Numero di SCC con grado entrante 0 (esclusa la partenza): "
                ++ show numeroZero
            putStrLn "=====\\n"
```

```

-----
-- LETTURA E VALIDAZIONE DEL GRAFO DA FILE
-----

{-
    Funzione per rimuovere il Byte Order Mark (BOM) da una stringa
    Parametro: Stringa da processare
-}
rimuoviBOM :: String -> String
rimuoviBOM ('\xFEFF':xs) = xs
rimuoviBOM xs = xs

{-
    Funzione per rimuovere l'andata a capo da una stringa Windows
    Parametro: Stringa da processare
-}
rimuoviCR :: String -> String
rimuoviCR xs
    | not (null xs) && last xs == '\r' = init xs
    | otherwise = xs

{-
    Funzione per normalizzare completamente una riga
    Parametro: Stringa da processare
-}
normalizzaRiga :: String -> String
normalizzaRiga = rimuoviCR . rimuoviBOM

{-
    Funzione che legge e valida un grafo orientato da file.

    Parametri:
    - nomeFile : percorso del file da leggere

    Il file deve contenere almeno due righe:
    1) Lista dei vertici (es: [1,2,3])
    2) Lista degli archi (es: [(1,2),(2,3)])

    Restituisce:
    - Just (vertici, archi) se parsing e validazione hanno successo
    - Nothing se si verifica un errore
-}
leggiGrafoDaFile :: FilePath -> IO (Maybe ([Int], [(Int, Int)]))
leggiGrafoDaFile nomeFile = do
    setLocaleEncoding utf8

    h <- openFile nomeFile ReadMode
    hSetEncoding h utf8

    contenuto <- hGetContents h

```

```

let righe = map normalizzaRiga (lines contenuto)

case righe of
  vLine:aLine:_ -> parseGrafo vLine aLine
  _ -> errore "Errore: il file deve contenere almeno due righe."

{-
  Funzione che esegue il parsing testuale delle due righe lette dal file.

  Parametri:
  - rVertici : stringa contenente la lista dei vertici
  - rArchi   : stringa contenente la lista degli archi
-}
parseGrafo :: String -> String -> IO (Maybe ([Int], [(Int, Int)]))
parseGrafo rVertici rArchi =
  case (reads rVertici, reads rArchi) of
    ([(vertici, ""), [(archi, "")]] ->
      validaGrafo vertici archi
    _ ->
      errore ("Errore: formato non valido. " ++
        "Verificare parentesi, virgole e struttura delle liste.")

{-
  Funzione che verifica la validità semantica del grafo.

  Parametri:
  - vertici : lista dei vertici del grafo
  - archi   : lista degli archi orientati del grafo

  Controlli effettuati:
  - lista vertici non vuota
  - assenza di vertici duplicati
  - archi che usano solo vertici esistenti
-}
validaGrafo :: [Int] -> [(Int, Int)] -> IO (Maybe ([Int], [(Int, Int)]))
validaGrafo vertici archi
  | null vertici =
    errore ("Errore: la lista dei vertici è vuota. " ++
      "Il grafo deve contenere almeno un vertice.")
  | not (verticiSenzaDuplicati vertici) =
    errore ("Errore: la lista dei vertici contiene duplicati. " ++
      "Ogni vertice deve comparire una sola volta.")
  | not (archiValidi vertici archi) =
    errore ("Errore: esistono archi che utilizzano " ++
      "vertici non presenti nella lista dei vertici.")
  | otherwise =
    return (Just (vertici, archi))

{-
  Funzione che verifica l'assenza di vertici duplicati.

```

```

    Parametri:
    - vertici : lista dei vertici del grafo

    Restituisce:
    - True se tutti i vertici sono distinti
    - False se esiste almeno un duplicato
-}
verticiSenzaDuplicati :: [Int] -> Bool
verticiSenzaDuplicati vertici =
    length vertici == length (nub vertici)

{-
    Funzione che controlla che ogni arco utilizzi solo vertici esistenti.

    Parametri:
    - vertici : lista dei vertici ammessi
    - archi   : lista degli archi del grafo
-}
archiValidi :: [Int] -> [(Int, Int)] -> Bool
archiValidi vertici =
    all (\(x, y) -> x `elem` vertici && y `elem` vertici)

{-
    Funzione che stampa un messaggio di errore e restituisce Nothing.
-}
errore :: String -> IO (Maybe a)
errore msg = putStrLn msg >> return Nothing

-----
-- FUNZIONE DI ACQUISIZIONE DEL VERTICE
-----

{-
    Funzione che acquisisce da tastiera un vertice valido.

    Parametri:
    - vertici : lista dei vertici del grafo

    Continua a chiedere input finché l'utente non inserisce
    un intero presente nella lista dei vertici.
-}
acquisisciVertice :: [Int] -> IO Int
acquisisciVertice vertici = do
    putStrLn $
        "Inserisci il vertice di partenza (tra " ++ show vertici ++ "): "
    hFlush stdout
    input <- getLine
    case reads input :: [(Int, String)] of
        [(v, _)] | v `elem` vertici -> return v
        _ -> do

```

```

        putStrLn ("Vertice non valido: " ++
                  "inserire un numero presente nella lista dei vertici.")
        acquisisciVertice vertici

-----

-- FUNZIONI DI BASE SUI GRAFI
-----

{-
  Funzione che restituisce tutti i vertici adiacenti ad un vertice dato.
  Parametri:
  - v      : vertice di partenza
  - archi  : lista degli archi del grafo
-}
verticiAdiacenti :: Int -> [(Int, Int)] -> [Int]
verticiAdiacenti v archi = [y | (x, y) <- archi, x == v]

{-
  Funzione che inverte la direzione di tutti gli archi del grafo.
  Parametri:
  - archi  : lista degli archi del grafo

  Restituisce:
  - archi invertiti
-}
invertiArchi :: [(Int, Int)] -> [(Int, Int)]
invertiArchi [] = []
invertiArchi ((x, y) : xs) = (y, x) : invertiArchi xs

-----

-- FUNZIONI DI COMBINAZIONE PER LA VISITA IN PROFONDITA'
-----

{-
  Funzione che inserisce il vertice in coda (post-order).
  Parametri:
  - vertice
  - risultato

  Restituisce:
  - risultato aggiornato
-}
aggiungiInCoda :: Int -> [Int] -> [Int]
aggiungiInCoda v res = res ++ [v]

{-
  Funzione che inserisce il vertice in test (pre-order).
  Parametri:
  - vertice
  - risultato
-}

```

```

    Restituisce:
    - risultato aggiornato
-}
aggiungiInTesta :: Int -> [Int] -> [Int]
aggiungiInTesta v res = v : res

-----

-- VISITA IN PROFONDITA' GENERICA
-----

{-
    Funzione che implementa una visita in profondità generica parametrizzata.

    Parametri:
    - combina : funzione di combinazione dei risultati
    - v       : vertice corrente
    - archi    : lista archi
    - visitati : lista vertici già visitati
-}
visitaInProfondita :: (Int -> [Int] -> [Int]) -> Int -> [(Int, Int)] -> [Int] -> ([Int], [Int])
visitaInProfondita combina v archi visitati
    | v `elem` visitati = (visitati , [])
    | otherwise =
        let visitati' = v : visitati
            (visitatiFinali, risultatiFigli) =
                foldl visita (visitati', []) (verticiAdiacenti v archi)
        in (visitatiFinali, combina v risultatiFigli)
where
    visita (vis, res) u =
        let (vis', res') = visitaInProfondita combina u archi vis
        in (vis', res ++ res')

-----

-- VISITA SPECIALIZZATE
-----

{-
    Funzione che implementa la visita in profondità
    per il calcolo dell'ordine di fine.
    Parametri:
    - vertice di partenza
    - archi
    - visitati

    Restituisce:
    - visitati
    - ordine di fine
-}
visitaInProfonditaOrdineFine :: Int -> [(Int, Int)] -> [Int] -> ([Int], [Int])
visitaInProfonditaOrdineFine = visitaInProfondita aggiungiInCoda

```

```

{-
  Funzione che implementa la visita in profondità per
  la costruzione delle componenti fortemente connesse.
  Parametri:
  - vertice di partenza
  - archi
  - visitati

  Restituisce:
  - visitati
  - scc
-}
visitaInProfonditaComponente :: Int -> [(Int, Int)] -> [Int] -> ([Int], [Int])
visitaInProfonditaComponente = visitaInProfondita aggiungiInTesta

```

```

-----
-- ORDINE DI FINE
-----

```

```

{-
  Funzione che calcola l'ordine di fine globale del grafo.
  Parametri:
  - vertici
  - archi

  Restituisce:
  - ordine
-}
ordineDiFine :: [Int] -> [(Int, Int)] -> [Int]
ordineDiFine vertici archi =
  snd $
    foldl visitaGlobale ([], []) vertici
  where
    visitaGlobale (vis, ord) v =
      let (vis', ord') = visitaInProfonditaOrdineFine v archi vis
      in (vis', ord ++ ord')

```

```

-----
-- KOSARAJU
-----

```

```

{-
  Funzione che implementa l'algoritmo di Kosaraju
  per componenti fortemente connesse.
  Parametri:
  - vertici
  - archi

  Restituisce:
  - componenti
-}

```

```

kosaraju :: [Int] -> [(Int, Int)] -> [[Int]]
kosaraju vertici archi =
    componenti
  where
    ordine          = reverse (ordineDiFine vertici archi)
    archiInvertiti = invertiArchi archi
    componenti =
        snd $
            foldl costruisci ([], []) ordine

    costruisci (vis, comps) v
        | v `elem` vis = (vis, comps)
        | otherwise =
            let (vis', comp) = visitaInProfonditaComponente v archiInvertiti vis
            in (vis', comps ++ [comp])

-----
-- GRAFO COMPRESSO
-----

{-
    Funzione che restituisce l'indice della componente
    fortemente connessa contenente un vertice.
    Parametri:
    - vertice
    - componenti

    Restituisce:
    - indice
-}
indiceSCC :: Int -> [[Int]] -> Int
indiceSCC v sccs =
    case [i | (i, comp) <- zip [0 ..] sccs , v `elem` comp] of
        (i:_) -> i
        []     -> error ("Errore interno: vertice non trovato in alcuna SCC: " ++ show v)

{-
    Funzione che costruisce il grafo compresso
    delle componenti fortemente connesse.
    Parametri:
    - componenti
    - archi

    Restituisce:
    - archi
-}
comprimiGrafo :: [[Int]] -> [(Int, Int)] -> [(Int, Int)]
comprimiGrafo sccs archi =
    nub
        [ (i, j)
        | (x, y) <- archi

```

```

    , let i = indiceSCC x sccs
    , let j = indiceSCC y sccs
    , i /= j
  ]

{-
  Funzione che calcola il grado entrante di
  una componente fortemente connessa.
  Parametri:
  - indice
  - archi

  Restituisce:
  - grado
-}

gradoEntrante :: Int -> [(Int, Int)] -> Int
gradoEntrante c archi =
  length [(x, y) | (x, y) <- archi , y == c]

{-
  Funzione che conta le componenti fortemente connesse
  con grado entrante zero (esclusa quella di partenza).
  Parametri:
  - vertice di partenza
  - componenti
  - archi

  Restituisce:
  - numero
-}

contaSCCConGradoZero :: Int -> [[Int]] -> [(Int, Int)] -> Int
contaSCCConGradoZero v sccs archi =
  length
    [ i
    | i <- [0 .. length sccs - 1]
    , i /= indiceSCC v sccs
    , gradoEntrante i archi == 0
    ]

```

File: connessioniGraf.hs

4.2 Implementazione in Prolog

```
% #####
% #          Corso di Programmazione Logica e Funzionale          #
% #          Progetto per la sessione invernale A.A 2025/2026      #
% #                      di Andrea Pedini                          #
% #                      Matricola: 322918                         #
% #                      e Matteo Fraternali                      #
% #                      Matricola: 316637                         #
% #####

% Scrivere un programma Haskell e un programma Prolog che leggano
% da file e stampino a schermo i seguenti dati di input:
% * Un insieme di numeri interi
% * Una insieme di coppie di numeri interi
% Successivamente il programma dovrà:
% * Costruire un grafo orientato i cui vertici sono costituiti
%   dalla lista di interi letta da file e gli archi
%   sono rappresentati dalle coppie di interi letti da file.
% * Costruire un nuovo grafo orientato in cui i vertici sono le componenti
%   fortemente connesse del grafo iniziale rappresentate da un singolo vertice
%   scelto arbitrariamente tra i vertici di una componente fortemente connessa e
%   gli archi sono i collegamenti tra le componenti fortemente connesse distinte.
% * Acquisire un numero intero tra i vertici del grafo a rappresentare il vertice di
%   partenza e stampare a schermo il numero di componenti fortemente
%   connesse diverse dalla componente contenente il vertice di partenza che
%   hanno grado entrante uguale a 0.

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
%% MAIN
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

/*
Predicato che avvia il programma leggendo il grafo dal file e gestendo il risultato della
lettura.
Parametri:
- Nessuno
*/
main :-
    leggi_grafo_sicuro('input.txt', Risultato),
    gestisci_risultato(Risultato).

/*
Predicato che gestisce il risultato della lettura e validazione del file di input.
Parametri:
- Risultato: può assumere il valore errore oppure ok(Grafo)
*/
gestisci_risultato(errore) :-
    nl, stampa_separatore,
    write('Errore nel file di input. '), nl,
    write('Controllare formato, duplicati e contenuto del grafo. '), nl,
```

```

        stampa_separatore, nl, !, fail.

gestisci_risultato(ok(Grafo)) :-
    esegui_programma(Grafo).

/*
Predicato che coordina l'esecuzione del programma stampando il grafo, calcolando le SCC e
mostrando i risultati finali.
Parametri:
- Grafo: struttura grafo(Vertici, Archi) rappresentante il grafo orientato
*/
esegui_programma(Grafo) :-
    vertici(Grafo, Vertici),
    archi(Grafo, Archi),
    kosaraju(Grafo, SCCs),

    stampa_separatore,
    write('          GRAFO LETTO DA FILE          '), nl,
    stampa_riga,
    write('Vertici: '), write(Vertici), nl,
    write('Archi:   '), write(Archi), nl,

    nl, stampa_separatore,
    write('          COMPONENTI FORTEMENTE CONNESSE '), nl,
    stampa_riga,
    stampa_scc_numerate(SCCs, 0),

    nl, stampa_separatore,
    write('          GRAFO COMPRESSO'), nl,
    stampa_riga,

    leggi_vertice_valido(Vertici, VerticeScelto),
    scc_di_vertice(VerticeScelto, SCCs, SCCPartenza),

    findall(S, (membro(S, SCCs), S \= SCCPartenza,
                grado_entrante(Grafo, SCCs, S, 0)), SCCZeroIn),
    length(SCCZeroIn, Conteggio),

    nl, stampa_separatore,
    write('Numero di SCC con indegree 0 (esclusa partenza): '),
    write(Conteggio), nl,
    stampa_separatore.

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
%% LETTURA E VALIDAZIONE DEL GRAFO
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

/*
Predicato che legge un grafo da file effettuando controlli su formato, duplicati e coerenza
degli archi.
Parametri:

```

```

- File: nome del file da cui leggere il grafo
- Risultato: ok(grafo(Vertici,Archi)) se valido, altrimenti errore
*/
leggi_grafo_sicuro(File, ok(grafo(Vertici, Archi))) :-
    catch(open(File, read, Stream), _, fail),

    leggi_termine_sicuro(Stream, Vertici),
    is_list(Vertici),
    Vertici \= [],
    lista_vertici_valida(Vertici),
    vertici_senza_duplicati(Vertici),

    leggi_termine_sicuro(Stream, Archi),
    is_list(Archi),
    lista_archi_valida(Archi),

    close(Stream),
    archi_validi(Archi, Vertici), !.

leggi_grafo_sicuro(_, errore).

/*
Predicato che verifica che una lista di vertici non contenga elementi duplicati.
Parametri:
- ListaVertici: lista di vertici da controllare
*/
vertici_senza_duplicati([]).
vertici_senza_duplicati([H|T]) :-
    \+ membro(H, T),
    vertici_senza_duplicati(T).

/*
Predicato che legge un termine Prolog da uno stream intercettando eventuali eccezioni.
Parametri:
- Stream: stream di input
- Termine: termine Prolog letto dallo stream
*/
leggi_termine_sicuro(Stream, Termine) :-
    leggi_linea(Stream, Line),
    atom_concat(Line, '.', LineConPunto),
    catch(read_from_atom(LineConPunto, Termine), _, fail).

/*
Predicato che verifica che una lista contenga esclusivamente numeri interi.
Parametri:
- ListaVertici: lista da validare
*/
lista_vertici_valida([]).
lista_vertici_valida([H|T]) :-
    integer(H),
    lista_vertici_valida(T).

```

```

/*
Predicato che verifica che una lista contenga solo coppie di interi rappresentanti archi.
Parametri:
- ListaArchi: lista di coppie (X,Y)
*/
lista_archi_valida([]).
lista_archi_valida([(X,Y)|T]) :-
    integer(X),
    integer(Y),
    lista_archi_valida(T).

/*
Predicato che verifica che ogni arco utilizzi solo vertici presenti nella lista dei vertici.
Parametri:
- Archi: lista degli archi
- Vertici: lista dei vertici del grafo
*/
archi_validi([], _).
archi_validi([(X,Y)|Resto], Vertici) :-
    membro(X, Vertici),
    membro(Y, Vertici),
    archi_validi(Resto, Vertici).

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
%% LETTURA DA STREAM
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

/*
Predicato che legge una riga da uno stream e la restituisce come atomo.
Parametri:
- Stream: stream di input
- Linea: atomo contenente la riga letta
*/
leggi_linea(Stream, Linea) :-
    get_char(Stream, Char),
    leggi_linea_auxiliario(Stream, Char, Caratteri),
    atom_chars(Linea, Caratteri).

/*
Predicato che legge carattere per carattere fino a newline o fine file.
Parametri:
- Stream: stream di input
- Char: carattere corrente letto
- ListaCaratteri: lista dei caratteri letti
*/
leggi_linea_auxiliario(_, end_of_file, []) :- !.
leggi_linea_auxiliario(_, '\n', []) :- !.
leggi_linea_auxiliario(Stream, Char, [Char|Resto]) :-
    get_char(Stream, Next),
    leggi_linea_auxiliario(Stream, Next, Resto).

```

```
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
```

```
%% INPUT UTENTE
```

```
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
```

```
/*
```

```
Predicato che richiede all'utente un vertice valido presente nel grafo.
```

```
Parametri:
```

- Vertici: lista dei vertici del grafo
- Vertice: vertice scelto e validato

```
*/
```

```
leggi_vertice_valido(Vertici, Vertice) :-
```

```
    write('Inserisci il vertice di partenza (tra '),  
    write(Vertici), write('): '),  
    leggi_numero_valido(N),  
    (membro(N, Vertici) -> Vertice = N  
    ; write('Vertice non valido! Riprova.'), nl,  
      leggi_vertice_valido(Vertici, Vertice)).
```

```
/*
```

```
Predicato che legge da input standard un numero intero valido.
```

```
Parametri:
```

- N: numero intero letto e validato

```
*/
```

```
leggi_numero_valido(N) :-
```

```
    leggi_linea(user_input, Line),  
    (leggi_e_valida_numero(Line, N) -> true  
    ; write('Input non valido! Inserisci un numero intero.'), nl,  
      leggi_numero_valido(N)).
```

```
/*
```

```
Predicato che verifica che una stringa rappresenti esattamente un numero intero.
```

```
Parametri:
```

- Line: atomo contenente l'input dell'utente
- N: numero intero risultante

```
*/
```

```
leggi_e_valida_numero(Line, N) :-
```

```
    atom_concat(Line, '.', LineConPunto),  
    catch(read_from_atom(LineConPunto, Termine), _, fail),  
    numero_valido(Termine, N).
```

```
/*
```

```
Predicato che controlla che un termine sia un numero intero puro.
```

```
Parametri:
```

- Termine: termine da verificare
- N: numero intero valido risultante

```
*/
```

```
numero_valido(N, N) :- integer(N).
```

```
numero_valido(Termine, _) :-
```

```
    \+ integer(Termine),  
    write('Errore: '), write(Termine), write('" non e'' un numero intero.'), nl,
```

```

fail.

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
%% STAMPA
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

/*
Predicato che stampa una linea grafica di separazione lunga.
Parametri:
- Nessuno
*/
stampa_separatore :- write('====='), nl.

/*
Predicato che stampa una linea grafica di separazione corta.
Parametri:
- Nessuno
*/
stampa_riga :- write('-----'), nl.

/*
Predicato che stampa le componenti fortemente connesse numerandole progressivamente.
Parametri:
- ListaSCC: lista delle componenti fortemente connesse
- Indice: numero progressivo della componente
*/
stampa_scc_numerate([], _).
stampa_scc_numerate([S|Resto], Indice) :-
    write('SCC '), write(Indice), write(': '), write(S), nl,
    Next is Indice + 1,
    stampa_scc_numerate(Resto, Next).

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
%% PREDICATI DI BASE SUL GRAFO
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

/*
Predicato che verifica l'appartenenza di un elemento a una lista.
Parametri:
- X: elemento da cercare
- Lista: lista in cui cercare
*/
membro(X, [X|_]).
membro(X, [_|Resto]) :- membro(X, Resto).

/*
Predicato che estrae la lista dei vertici da una struttura grafo.
Parametri:
- Grafo: termine grafo(Vertici,Archi)
- Vertici: lista dei vertici estratta
*/

```

```

vertici(grafo(N,_), N).

/*
Predicato che estrae la lista degli archi da una struttura grafo.
Parametri:
- Grafo: termine grafo(Vertici,Archi)
- Archi: lista degli archi estratta
*/
archi(grafo(_,A), A).

/*
Predicato che verifica l'esistenza di un arco orientato tra due vertici.
Parametri:
- Grafo: struttura del grafo
- X: vertice di partenza
- Y: vertice di arrivo
*/
adiacente(grafo(_,A), X, Y) :-
    membro((X,Y), A).

/*
Predicato che restituisce la lista dei vertici adiacenti a un vertice dato.
Parametri:
- Grafo: struttura del grafo
- Vertice: vertice di partenza
- Adiacenti: lista dei vertici raggiungibili
*/
adiacenti(Grafo, Vertice, Adiacenti) :-
    findall(Y, adiacente(Grafo, Vertice, Y), Adiacenti).

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
%% VISITA IN PROFONDITÀ
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

/*
Predicato che implementa la visita in profondità parametrizzata con una strategia di
combinazione.
Parametri:
- Combina: predicato di combinazione dei risultati
- Grafo: struttura del grafo
- Vertice: vertice corrente
- Visitati: lista dei vertici già visitati
- VisitatiFinali: lista finale dei vertici visitati
- Risultato: risultato aggregato della visita
*/
visitaInProfondita(_, _, Vertice, Visitati, Visitati, []) :-
    membro(Vertice, Visitati), !.

visitaInProfondita(Combina, Grafo, Vertice, Visitati, VisitatiFinali, Risultato) :-
    \+ membro(Vertice, Visitati),
    adiacenti(Grafo, Vertice, Vicini),

```

```

    visitaInProfondita_lista(Combina, Grafo, Vicini,
                             [Vertice|Visitati], VisitatiParziali, RisultatiFigli),
    call(Combina, Vertice, RisultatiFigli, Risultato),
    VisitatiFinali = VisitatiParziali.

/*
Predicato che applica la visita in profondità a una lista di vertici.
Parametri:
- Combina: predicato di combinazione
- Grafo: struttura del grafo
- ListaVertici: lista di vertici da visitare
- Visitati: lista dei vertici già visitati
- VisitatiFinali: lista finale dei vertici visitati
- Risultato: risultato aggregato complessivo
*/
visitaInProfondita_lista(_, _, [], Visitati, Visitati, []).
visitaInProfondita_lista(Combina, Grafo, [H|T], Visitati, VisitatiFinali, Risultato) :-
    visitaInProfondita(Combina, Grafo, H, Visitati, Visitati1, R1),
    visitaInProfondita_lista(Combina, Grafo, T, Visitati1, VisitatiFinali, R2),
    append(R1, R2, Risultato).

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
%% VISITE SPECIALIZZATE
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

/*
Predicato che esegue una visita in profondità costruendo l'ordine di completamento.
Parametri:
- Grafo: struttura del grafo
- Vertice: vertice di partenza
- Visitati: lista dei vertici già visitati
- VisitatiFinali: lista finale dei vertici visitati
- Ordine: lista dei vertici in ordine di completamento
*/
visitaInProfondita_ordine(Grafo, Vertice, Visitati, VisitatiFinali, Ordine) :-
    visitaInProfondita(combina_fine, Grafo, Vertice, Visitati, VisitatiFinali, Ordine).

/*
Predicato che esegue una visita in profondità per costruire una componente fortemente connessa.
Parametri:
- Grafo: struttura del grafo
- Vertice: vertice di partenza
- Visitati: lista dei vertici già visitati
- VisitatiFinali: lista finale dei vertici visitati
- Componente: lista dei vertici appartenenti alla SCC
*/
visitaInProfondita_scc(Grafo, Vertice, Visitati, VisitatiFinali, Componente) :-
    visitaInProfondita(combina_testa, Grafo, Vertice, Visitati, VisitatiFinali, Componente).

/*
Predicato che esegue la visita in profondità sull'intero grafo.

```

```

Parametri:
- Grafo: struttura del grafo
- Ordine: lista dei vertici in ordine di completamento globale
*/
visitaInProfondita_grafo(Grafo, Ordine) :-
    vertici(Grafo, Vertici),
    visitaInProfondita_lista(combina_fine, Grafo, Vertici, [], _, Ordine).

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
%% STRATEGIE DI COMBINAZIONE
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

/*
Predicato che inserisce un vertice in coda a una lista.
Parametri:
- N: vertice da inserire
- Lista: lista esistente
- Risultato: lista con N aggiunto in coda
*/
combina_fine(N, Lista, Risultato) :-
    append(Lista, [N], Risultato).

/*
Predicato che inserisce un vertice in testa a una lista.
Parametri:
- N: vertice da inserire
- Lista: lista esistente
- Risultato: lista con N aggiunto in testa
*/
combina_testa(N, Lista, [N|Lista]).

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
%% GRAFO TRASPOSTO
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

/*
Predicato che costruisce il grafo trasposto invertendo tutti gli archi.
Parametri:
- Grafo: grafo originale
- GrafoTrasposto: grafo con archi invertiti
*/
trasposto(grafo(Vertici, Archi), grafo(Vertici, ArchiTrasposti)) :-
    trasponi_archi(Archi, ArchiTrasposti).

/*
Predicato che inverte la direzione di ogni arco.
Parametri:
- Archi: lista originale di archi
- ArchiTrasposti: lista di archi invertiti
*/
trasponi_archi([], []).

```

```

trasponi_archi([(X,Y)|Resto], [(Y,X)|Trasposti]) :-
    trasponi_archi(Resto, Trasposti).

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
%% ALGORITMO DI KOSARAJU
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

/*
Predicato che implementa l'algoritmo di Kosaraju per il calcolo delle componenti fortemente
connesse.
Parametri:
- Grafo: struttura del grafo
- SCCs: lista delle componenti fortemente connesse trovate
*/
kosaraju(Grafo, SCCs) :-
    visitaInProfondita_grafo(Grafo, Ordine),
    reverse(Ordine, OrdineInverso),
    trasposto(Grafo, GrafoTrasposto),
    kosaraju_visita(GrafoTrasposto, OrdineInverso, [], SCCs).

/*
Predicato che costruisce progressivamente l'elenco delle componenti fortemente connesse.
Parametri:
- Grafo: grafo trasposto
- ListaVertici: vertici ordinati per visita
- Visitati: lista dei vertici già assegnati a una SCC
- SCCs: lista finale delle componenti fortemente connesse
*/
kosaraju_visita(_, [], _, []).
kosaraju_visita(Grafo, [Vertice|Resto], Visitati, SCCs) :-
    membro(Vertice, Visitati), !,
    kosaraju_visita(Grafo, Resto, Visitati, SCCs).

kosaraju_visita(Grafo, [Vertice|Resto], Visitati, [SCC|Altre]) :-
    visitaInProfondita_scc(Grafo, Vertice, Visitati, Visitati1, SCC),
    kosaraju_visita(Grafo, Resto, Visitati1, Altre).

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
%% GRAFO COMPRESSO DELLE SCC
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

/*
Predicato che individua la componente fortemente connessa contenente un dato vertice.
Parametri:
- Vertice: vertice da cercare
- ListaSCC: lista delle componenti
- SCC: componente che contiene il vertice
*/
scc_di_vertice(Vertice, [S|_], S) :-
    membro(Vertice, S), !.
scc_di_vertice(Vertice, [_|Resto], S) :-

```

```

    scc_di_vertice(Vertice, Resto, S).

/*
Predicato che verifica l'esistenza di un arco tra due componenti fortemente connesse distinte.
Parametri:
- Grafo: struttura del grafo originale
- SCCs: lista delle componenti fortemente connesse
- S1: componente sorgente
- S2: componente destinazione
*/
arco_scc(Grafo, SCCs, S1, S2) :-
    archi(Grafo, Archi),
    membro((X,Y), Archi),
    scc_di_vertice(X, SCCs, S1),
    scc_di_vertice(Y, SCCs, S2),
    S1 \= S2.

/*
Predicato che calcola il grado entrante di una componente fortemente connessa nel grafo
compresso.
Parametri:
- Grafo: struttura del grafo originale
- SCCs: lista delle componenti fortemente connesse
- SCC: componente di cui calcolare il grado entrante
- Grado: numero di archi entranti nella componente
*/
grado_entrante(Grafo, SCCs, SCC, Grado) :-
    findall(1, arco_scc(Grafo, SCCs, _, SCC), Lista),
    length(Lista, Grado).

```

File: connessioniGrafì.pl

5 Testing del Programma

Testing del programma Haskell

Test 1

Vertici: 1,2,3,4,5

Archi: (1,2) - (2,3) - (3,4) - (4,5) - (5,1)

SCC: [1,5,4,3,2]

Vertice di partenza: 4

Risultato: 0

Test 2

Vertici: 1,2,3,4,5,6

Archi: (1,2) - (2,3) - (3,1) - (4,5)

SCC: [6] - [4] - [5] - [1,3,2]

Vertice di partenza: 4

Risultato: 2

Test 3

Vertici: 1,2,3,4,5,6

Archi: (1,2) - (1,3) - (2,4) - (3,4) - (4,5) - (4,6)

SCC: [1] - [3] - [2] - [4] - [6] - [5]

Vertice di partenza: 2

Risultato: 1

Test 4

Vertici: 1,2,3,4,5,6,7

Archi: (1,2) - (2,3) - (3,1) - (3,4) - (4,5) - (5,3) - (5,6) - (6,7)

SCC: [1,3,2,5,4] - [6] - [7]

Vertice di partenza: 5

Risultato: 0

Test 5

Vertici: 1,2,3,4,5,6

Archi: (1,2) - (2,3) - (3,3) - (4,5)

SCC: [6] - [4] - [5] - [1] - [2] - [3]

Vertice di partenza: 6

Risultato: 2

Test 6

Vertici: 1,2,3,4,5,6

Archi: (1,2) - (1,3) - (1,4) - (1,5) - (1,6)

SCC: [1] - [6] - [5] - [4] - [3] - [2]

Vertice di partenza: 3

Risultato: 1

Test 7

Vertici: 1,2,3,4,5,6,7,8

Archi: (1,2) - (2,3) - (3,1) - (4,5) - (5,6) - (6,4) - (6,7) - (7,8)

SCC: [4,6,5] - [7] - [8] - [1,3,2]

Vertice di partenza: 8

Risultato: 2

Test 8

Vertici: 1,2,3,4,5,6,7

Archi: (1,2) - (2,3) - (3,4) - (4,5) - (5,6) - (6,7) - (7,4)

SCC: [1] - [2] - [3] - [4,7,6,5]

Vertice di partenza: 7

Risultato: 1

Test 9

Vertici: 1,2,3,4,5,6,7,8

Archi: (1,2) - (2,3) - (3,1) - (3,4) - (4,5) - (5,3) - (5,6) - (6,7) - (7,6) - (7,8)

SCC: [1,3,2,5,4] - [6,7] - [8]

Vertice di partenza: 8

Risultato: 1

Test 10

Vertici: 1,2,3,4,5,6,7

Archi: (1,2) - (2,3) - (3,1) - (2,4) - (4,5) - (5,6) - (6,4) - (6,7)

SCC: [1,3,2] - [4,6,5] - [7]

Vertice di partenza: 6

Risultato: 1

Testing del programma Prolog

Test 1

Vertici: 1,2,3,4,5

Archi: (1,2) - (2,3) - (3,4) - (4,5) - (5,1)

SCC: [1,5,4,3,2]

Vertice di partenza: 4

Risultato: 0

Test 2

Vertici: 1,2,3,4,5,6

Archi: (1,2) - (2,3) - (3,1) - (4,5)

SCC: [6] - [4] - [5] - [1,3,2]

Vertice di partenza: 4

Risultato: 2

Test 3

Vertici: 1,2,3,4,5,6

Archi: (1,2) - (1,3) - (2,4) - (3,4) - (4,5) - (4,6)

SCC: [1] - [3] - [2] - [4] - [6] - [5]

Vertice di partenza: 2

Risultato: 1

Test 4

Vertici: 1,2,3,4,5,6,7

Archi: (1,2) - (2,3) - (3,1) - (3,4) - (4,5) - (5,3) - (5,6) - (6,7)

SCC: [1,3,2,5,4] - [6] - [7]

Vertice di partenza: 5

Risultato: 0

Test 5

Vertici: 1,2,3,4,5,6

Archi: (1,2) - (2,3) - (3,3) - (4,5)

SCC: [6] - [4] - [5] - [1] - [2] - [3]

Vertice di partenza: 6

Risultato: 2

Test 6

Vertici: 1,2,3,4,5,6

Archi: (1,2) - (1,3) - (1,4) - (1,5) - (1,6)

SCC: [1] - [6] - [5] - [4] - [3] - [2]

Vertice di partenza: 3

Risultato: 1

Test 7

Vertici: 1,2,3,4,5,6,7,8

Archi: (1,2) - (2,3) - (3,1) - (4,5) - (5,6) - (6,4) - (6,7) - (7,8)

SCC: [4,6,5] - [7] - [8] - [1,3,2]

Vertice di partenza: 8

Risultato: 2

Test 8

Vertici: 1,2,3,4,5,6,7

Archi: (1,2) - (2,3) - (3,4) - (4,5) - (5,6) - (6,7) - (7,4)

SCC: [1] - [2] - [3] - [4,7,6,5]

Vertice di partenza: 7

Risultato: 1

Test 9

Vertici: 1,2,3,4,5,6,7,8

Archi: (1,2) - (2,3) - (3,1) - (3,4) - (4,5) - (5,3) - (5,6) - (6,7) - (7,6) - (7,8)

SCC: [1,3,2,5,4] - [6,7] - [8]

Vertice di partenza: 8

Risultato: 1

Test 10

Vertici: 1,2,3,4,5,6,7

Archi: (1,2) - (2,3) - (3,1) - (2,4) - (4,5) - (5,6) - (6,4) - (6,7)

SCC: [1,3,2] - [4,6,5] - [7]

Vertice di partenza: 6

Risultato: 1